Инженерный разбор системы учета токенов, расчета стоимости и контроля бюджетов в DevCrew

1. Архитектурное решение: Финансовый контур и контроль бюджетов

При разработке автономных мультиагентных систем без внешнего надзора существует высокий риск неконтролируемого расхода средств:

1. **Бесконечные циклы вызовов функций (Infinite Tool-Calling Loops):** Агент может попасть в зацикливание при попытке исправить ошибку сборки или некорректном считывании файлов.
2. **Экспоненциальный рост контекста (Context Window Inflation):** С каждым шагом (Turn) размер истории диалога растет. На 30-й итерации один вызов `gpt-4o` может стоить в 10–20 раз дороже, чем на 1-й.
3. **Отсутствие барьеров в подписках:** Каждый вызов LLM — это прямая микротранзакция с баланса OpenAI API.

Ключевые принципы финансового контура DevCrew:

- **Cents-based Accounting (Учет в центах):** Избегание типа `float` / `double` в PostgreSQL из-за ошибок округления чисел с плавающей запятой в JavaScript (`0.1 + 0.2 !== 0.3`). Все лимиты и расходы хранятся как целые числа (`integer`), где `$1.00 = 100` центов, а `$50.00 = 5000` центов.
- **Pre-Execution Lock (Пре-валидация):** Агент **не имеет права** сделать сетевой запрос в OpenAI API или вызвать Tool, пока база данных не подтвердит: `spentCents < budgetCents` .
- **Exact Usage Billing (Атомарное списание):** Из объекта ответа OpenAI (`response.usage`) извлекаются фактические значения `prompt_tokens` и `completion_tokens` , конвертируются в центы и атомарно прибавляются в БД через `sql` инкремент.
- **Max Turns Limit Guard (Ограничение итераций):** Помимо денег, каждый агент ограничен по глубинным итерациям (Max Turns). При достижении порога оркестратор принудительно меняет `tool_choice` , обрывая бесконечную генерацию.

---

2. Матрица расхода токенов, лимитов и стоимости по ролям агентов

Среднестатистический расход токенов за **один полный успешный проход конвейера** (от тикета до PR) распределяется следующим образом:

Агент	Лимит итераций (`maxTurns`)	Средний входной контекст (`prompt_tokens`)	Средний выход (`completion_tokens`)	Средний расход токенов на задачу	Оценочная стоимость на задачу (в центах USD)*
Analyst	**20**	$\sim 15,000 - 45,000$	$\sim 2,000 - 5,000$	$\sim 50,000$	**$\sim 10¢ - 18¢$** ($\approx $0.10 - 0.18)
Developer	**60**	$\sim 80,000 - 250,000$	$\sim 8,000 - 25,000$	$\sim 300,000$	**$\sim 65¢ - 180¢$** ($\approx $0.65 - 1.80)
Reviewer	**20**	$\sim 30,000 - 90,000$	$\sim 2,000 - 6,000$	$\sim 100,000$	**$\sim 20¢ - 35¢$** ($\approx $0.20 - 0.35)
Tester	**15**	$\sim 15,000 - 30,000$	$\sim 1,500 - 3,500$	$\sim 35,000$	**$\sim 8¢ - 15¢$** ($\approx $0.08 - 0.15)
ИТОГО:	—	—	—	**$\sim 485,000$ токенов**	**$\sim $1.03 - 2.48** за успешный PR

**Расчет стоимости приведен на базе тарифов OpenAI GPT-4o ($2.50 / 1M input, $10.00 / 1M output).*

---

3. Программная реализация финансового контура и оркестрации

A. Модуль контроля бюджетов и фиксации затрат (`lib/agent-budget.ts`)

```
import { db } from "@/db";
import { agents } from "@/db/schema";
import { eq, and, sql } from "drizzle-orm";

// Тарифы OpenAI (в центах за 1 миллион токенов)
const MODEL_RATES_IN_CENTS: Record<string, { inputPerMillion: number; outputPerMillion:
number }> = {
```

```
  "gpt-4o": {
    inputPerMillion: 250,   // $2.50 USD = 250 cents
    outputPerMillion: 1000, // $10.00 USD = 1000 cents
  },
  "gpt-4o-mini": {
    inputPerMillion: 15,    // $0.15 USD = 15 cents
    outputPerMillion: 60,   // $0.60 USD = 60 cents
  },
};

/**
 * 1. ПРЕ-ВАЛИДАЦИЯ: Проверяет лимит бюджета перед отправкой запроса в OpenAI API.
 */
export async function assertAgentCanRun(workspaceId: string, agentRole: string):
Promise<void> {
  const [agentRecord] = await db
    .select({
      budgetCents: agents.budgetCents,
      spentCents: agents.spentCents,
    })
    .from(agents)
    .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));

  if (!agentRecord) {
    throw new Error(`[Budget Error] Агент ${agentRole} не зарегистрирован.`);
  }

  // Блокировка вызова при превышении бюджета
  if (agentRecord.spentCents >= agentRecord.budgetCents) {
    const spentFormatted = (agentRecord.spentCents / 100).toFixed(2);
    const budgetFormatted = (agentRecord.budgetCents / 100).toFixed(2);

    throw new Error(
      `[Budget Exceeded] Выполнение агента ${agentRole} заблокировано. ` +
      `Израсходовано $${spentFormatted} из установленного лимита $${budgetFormatted}.`
    );
  }
}

/**
 * 2. ФИКСАЦИЯ ЗАТРАТ: Конвертирует токены в центы и атомарно инкрементирует spentCents в
PostgreSQL.
 */
export async function recordAgentSpend(params: {
  workspaceId: string;
  agentRole: string;
  promptTokens: number;
  completionTokens: number;
  model: string;
}): Promise<number> {
  const { workspaceId, agentRole, promptTokens, completionTokens, model } = params;

  const costInCents = calculateTokenCostInCents(model, promptTokens, completionTokens);

  // Атомарный инкремент значения spentCents в PostgreSQL
  await db
    .update(agents)
    .set({
```

```
      spentCents: sql`${agents.spentCents} + ${costInCents}`,
      updatedAt: new Date(),
    })
    .where(and(eq(agents.workspaceId, workspaceId), eq(agents.role, agentRole)));

  return costInCents;
}

/**
 * Расчет стоимости токенов в центах
 */
function calculateTokenCostInCents(model: string, inputTokens: number, outputTokens:
number): number {
  const rates = MODEL_RATES_IN_CENTS[model] || MODEL_RATES_IN_CENTS["gpt-4o"];

  const inputCost = (inputTokens / 1_000_000) * rates.inputPerMillion;
  const outputCost = (outputTokens / 1_000_000) * rates.outputPerMillion;

  return Math.max(1, Math.round(inputCost + outputCost));
}
```

Б. Исполнительный оркестратор пошагового цикла (`lib/task-runs.ts`)

```
import { openai } from "@/lib/openai";
import { assertAgentCanRun, recordAgentSpend } from "@/lib/agent-budget";
import { getAgentToolsDefinition, executeAgentTool } from "@/lib/agent-tools";

export async function executeAgentTurnLoop(params: {
  taskId: string;
  runId: string;
  workspaceId: string;
  agentRole: "analyst" | "developer" | "reviewer" | "tester";
  systemPrompt: string;
  taskContext: string;
  maxTurns: number;
}) {
  const { taskId, runId, workspaceId, agentRole, systemPrompt, taskContext, maxTurns } =
params;

  const conversationHistory: any[] = [
    { role: "system", content: systemPrompt },
    { role: "user", content: taskContext },
  ];

  let currentTurn = 0;
  let stageCompleted = false;

  while (currentTurn < maxTurns && !stageCompleted) {
    currentTurn++;

    // ШАГ 1: Проверка финансового лимита перед каждым вызовом LLM
    await assertAgentCanRun(workspaceId, agentRole);

    const isLastTurn = currentTurn === maxTurns;
```

```
    const tools = getAgentToolsDefinition(agentRole);

    // ШАГ 2: Запрос к OpenAI API (на последнем шаге принудительный tool_choice)
    const response = await openai.chat.completions.create({
      model: "gpt-4o",
      messages: conversationHistory,
      tools: tools.length > 0 ? tools : undefined,
      tool_choice: isLastTurn
        ? { type: "function", function: { name: `submit_${agentRole}` } }
        : "auto",
    });

    const choice = response.choices[0];
    const message = choice.message;
    conversationHistory.push(message);

    // ШАГ 3: Фиксация расхода токенов и списывание средств
    if (response.usage) {
      const costCents = await recordAgentSpend({
        workspaceId,
        agentRole,
        promptTokens: response.usage.prompt_tokens,
        completionTokens: response.usage.completion_tokens,
        model: "gpt-4o",
      });

      console.log(
        `[Turn Billing] Step ${currentTurn}/${maxTurns} | ` +
        `Agent: ${agentRole} | Tokens: ${response.usage.total_tokens} | Cost: ${costCents}
¢`
      );
    }

    // ШАГ 4: Выполнение вызовов инструментов (Tool Calls)
    if (message.tool_calls && message.tool_calls.length > 0) {
      for (const toolCall of message.tool_calls) {
        if (toolCall.function.name === `submit_${agentRole}`) {
          stageCompleted = true;
        }

        const toolResult = await executeAgentTool(toolCall, agentRole);

        conversationHistory.push({
          role: "tool",
          tool_call_id: toolCall.id,
          content: JSON.stringify(toolResult),
        });
      }
    }
  }
}
```